

Project #4 – AddDutyCycle

Document reference: Thies, Nate.cmpe311.fall25.Project#4-AddDutyCycle

Date of publication: December 2025

LEAD ENGINEER:

Nate Thies, University of Maryland Baltimore County

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County

MD Safwan Zaman, Teaching Assistant, University of Maryland Baltimore County

HIGH-LEVEL DESCRIPTION

This project adds variable-duty-cycle motor control to the existing Cyclic Executive (CE) system implemented in Project #2. A push button is used to cycle the motor through predetermined PWM duty-cycle steps, implemented via Timer2 interrupt-driven waveform generation. The motor is powered externally using an 8×AA battery pack and controlled through an IRLZ34N N-channel MOSFET functioning as a low-side switch. This project demonstrates real-time embedded control, external-power hardware integration, and safe switching of inductive loads.

DESCRIPTION

The system extends the prior CE task manager with additional functionality that enables the user to adjust motor speed in discrete increments using a pushbutton. The Arduino produces an eight-step discrete PWM waveform (0/8, 2/8, 4/8, 6/8, 8/8, 6/8, 4/8, 2/8, repeat) by configuring Timer2 in CTC mode and executing the PWM logic within an interrupt service routine. The MOSFET controls the ground return path for an externally powered DC motor, with a 1N4004 diode suppressing inductive kickback. The CE continues running its existing LED blink and serial-input tasks concurrently, satisfying responsiveness and non-blocking constraints.

This design solves the problem of safely controlling a DC motor from the Arduino while protecting the microcontroller from the motor's higher voltage and inductive transients. It also demonstrates correct integration of ISR-based timing into a CE-based architecture.

REFERENCES

- Elegoo UNO R3 (ATmega328P) datasheet
 - IRLZ34N MOSFET datasheet
 - 1N4004 rectifier diode datasheet
 - Arduino Timer2 documentation
 - CMPE311 Project #2 – Task Management (Cyclic Executive)
-

DEFINITIONS AND ABBREVIATIONS

PWM – Pulse Width Modulation

ISR – Interrupt Service Routine

MOSFET – Metal-Oxide-Semiconductor Field Effect Transistor

CE – Cyclic Executive

Duty Cycle – Fraction of one PWM period for which the signal remains HIGH

CONVENTIONS

- **Must, shall, will** – mandatory requirement
- **May** – optional behavior
- **Informative** – explanatory content only
- All customer requirements begin with **C.#**
- All high-level technical requirements begin with **HL.#**
- All test requirements begin with **T.#**

CUSTOMER REQUIREMENTS

C.1 The user must be able to control the speed of the DC motor.

C.1.1 Speed control shall be performed through a single pushbutton.

C.2 The system shall step through discrete speed levels in a predictable pattern.

C.2.1 Steps shall follow the sequence: $0/8 \rightarrow 2/8 \rightarrow 4/8 \rightarrow 6/8 \rightarrow 8/8 \rightarrow 6/8 \rightarrow 4/8 \rightarrow 2/8 \rightarrow 0/8$.

C.3 The system must operate on an Elegoo/Arduino UNO R3 development board.

C.4 The system must remain responsive, with no blocking delays.

C.5 The system shall operate safely with an externally powered motor.

HIGH-LEVEL TECHNICAL REQUIREMENTS

HL.1 The system shall generate PWM on digital pin D9 using hardware Timer2.

HL.1.1 Timer2 shall operate at a PWM frame frequency of 250 Hz.

HL.2 The system shall use an IRLZ34N N-channel MOSFET as a low-side switch for the motor.

HL.2.1 The gate shall include a 330–470 Ω series resistor and ≥ 4.7 k Ω pull-down resistor.

HL.3 The motor shall be powered from an 8×AA battery pack, isolated from Arduino 5 V logic power.

HL.4 A 1N4004 diode shall be placed across the motor to suppress inductive voltage spikes.

HL.5 The embedded firmware shall run under a non-preemptive cyclic executive.

HL.5.1 Existing tasks from Project #2 (serial UI, LED blinks) shall remain fully functional.

HL.6 No `delay()` statements shall be used anywhere in the design.

TESTING AND VALIDATION REQUIREMENTS

T.1 Verify pushbutton operation.

T.1.1 System shall print correct duty-step values to Serial Monitor when the button is pressed.

T.2 Verify PWM generation on D9.

T.2.1 A test LED on D9 shall visibly change brightness at each duty step.

T.3 Verify motor operation with direct battery-power testing (informative).

T.4 Validate MOSFET switching.

T.4.1 Motor shall turn off at 0/8 duty and increase speed as duty steps increase.

T.5 Verify CE responsiveness.

T.5.1 LED blink tasks must continue operating without interruption during motor PWM.

DESIGN

Equipment Required

- Elegoo/Arduino UNO R3
 - IRLZ34N N-channel MOSFET
 - 1N4004 diode
 - DC Motor (3–12 V)
 - 8×AA battery pack (≈ 12 V)
 - 330–470 Ω resistor (MOSFET gate series)
 - 4.7 k Ω resistor (MOSFET gate pulldown)
 - Pushbutton
 - Breadboard and jumper wires
-

Circuit Design (Text-Only Schematic Description)

Power Path

- Battery pack positive (+12 V) connects to the motor's positive terminal.
- Motor negative terminal connects to the MOSFET Drain (middle pin).
- MOSFET Source (right pin, when facing text) connects to ground.

Gate Drive

- Arduino D9 connects to MOSFET Gate (left pin) through a 330–470 Ω resistor.
- A 4.7 k Ω resistor connects Gate → Ground to ensure the MOSFET remains OFF during reset.

Inductive Protection

- A 1N4004 diode is placed across the motor terminals.
 - Cathode (striped) → motor positive (battery +)
 - Anode (non-striped) → motor negative (Drain)

Grounding

- Battery negative → motor/MOSFET ground → Arduino GND.
- All grounds must be common for the gate voltage to reference correctly.

Pushbutton

- One side → D4
- Other side → Ground
- Internal pull-up (`INPUT_PULLUP`) handles logic-high idle state.

Code

None

```
/*
Project #2 – Task Management (Cyclic Executive)
Platform: Elegoo/Arduino UNO R3 (ATmega328P)
Serial Monitor: 9600 baud

Summary:
- Implements a non-preemptive, round-robin cyclic executive
(CE)
  using a function-pointer task array.
```

```

- Tasks:
    1) taskSerialInput : non-blocking serial line reader +
    UI state machine
    2) taskBlinkLED1    : toggles LED1 according to its
    period
    3) taskBlinkLED2    : toggles LED2 according to its
    period
    - No delay(); all timing uses millis().
    - User-entered interval (ms) is treated as the FULL period;
    each
        ON and OFF lasts interval/2 milliseconds (matches P1
    dialog).

Usage (Serial):
    > What LED? (1 or 2)
    > 2
    > What interval (in msec)?
    > 600
    => LED2 blinks ~300 ms ON, ~300 ms OFF
*/

#include <Arduino.h>

// ----- Pins
// -----
static const uint8_t LED_PIN_1 = 2; // LED1 on D2
static const uint8_t LED_PIN_2 = 3; // LED2 on D3

// ----- Timing Model
// -----
// The "period" is the full blink period in milliseconds.
// Each LED toggles every halfPeriod = max(1, period/2).

```

```

// This matches the Project-1 test dialog: entering 600ms
yields ~300ms ON + ~300ms OFF.

static volatile unsigned long periodMs_LED1 = 1000;  //
default 1s period
static volatile unsigned long periodMs_LED2 = 1000;  //
default 1s period

// Internal toggling state
static unsigned long lastToggle_LED1 = 0;
static unsigned long lastToggle_LED2 = 0;
static bool state_LED1 = LOW;
static bool state_LED2 = LOW;

// ----- Serial UI State
-----

// Non-blocking line input buffer
static const size_t LINE_MAX = 32;
static char lineBuf[LINE_MAX];
static size_t lineLen = 0;

enum InputState : uint8_t {
    ASK_LED = 0,
    ASK_INTERVAL = 1
};

static InputState uiState = ASK_LED;
static int selectedLED = -1; // 1 or 2 after a valid LED entry

// ----- Helpers
-----

```

```

static inline unsigned long halfPeriodClamp(unsigned long
period) {
    // Ensure at least 1 ms half-period to avoid zero-interval
edge cases.
    unsigned long hp = period / 2;
    return (hp == 0 ? 1UL : hp);
}

static void promptAskLED() {
    Serial.println(F("What LED? (1 or 2)"));
}

static void promptAskInterval() {
    Serial.println(F("What interval (in msec)?"));
}

static bool isAllDigits(const char* s) {
    if (!s || !*s) return false;
    for (const char* p = s; *p; ++p) {
        if (*p < '0' || *p > '9') return false;
    }
    return true;
}

// Process a completed input line (no CR/LF included).
static void processLine(const char* line) {
    switch (uiState) {
        case ASK_LED: {
            if ((strcmp(line, "1") == 0) || (strcmp(line, "2") ==
0)) {
                selectedLED = atoi(line);
                uiState = ASK_INTERVAL;
            }
        }
    }
}

```

```

        promptAskInterval();
    } else {
        Serial.println(F("Invalid LED. Enter 1 or 2.));
        promptAskLED();
    }
} break;

case ASK_INTERVAL: {
    if (!isAllDigits(line)) {
        Serial.println(F("Invalid interval. Must be a positive
integer in milliseconds.));
        promptAskInterval();
        return;
    }
    // Parse and clamp (0 is invalid)
    unsigned long val = strtoul(line, nullptr, 10);
    if (val == 0) {
        Serial.println(F("Invalid interval. Must be > 0.));
        promptAskInterval();
        return;
    }

    if (selectedLED == 1) {
        periodMs_LED1 = val;
        Serial.print(F("LED1 period set to "));
        Serial.print(periodMs_LED1);
        Serial.println(F(" ms (ON/OFF ~half each).));
    } else if (selectedLED == 2) {
        periodMs_LED2 = val;
        Serial.print(F("LED2 period set to "));
        Serial.print(periodMs_LED2);
        Serial.println(F(" ms (ON/OFF ~half each).));
    }
}

```

```

    } else {
        // Should never happen; reset UI gracefully
        Serial.println(F("Internal state error; resetting
prompt."));
    }

    // Reset for next cycle
    uiState = ASK_LED;
    selectedLED = -1;
    promptAskLED();
} break;
}
}

// ----- Tasks
-----

// Task 1: Non-blocking serial input reader + UI state
machine.
// - Accumulates characters only when available.
// - Handles Backspace/Delete, ignores CR, ends lines on LF.
// - Never blocks; safe inside cyclic executive.
void taskSerialInput(void) {
    while (Serial.available() > 0) {
        char c = (char)Serial.read();

        if (c == '\r') {
            // Ignore CR; wait for LF to terminate line
            continue;
        } else if (c == '\n') {
            // End of line -> process
            lineBuf[lineLen] = '\0';
            if (lineLen > 0) {

```

```

        processLine(lineBuf);
    }
    lineLen = 0; // reset buffer
} else if (c == 8 || c == 127) { // Backspace or Delete
    if (lineLen > 0) {
        lineLen--;
    }
} else {
    // Accept visible characters up to buffer limit
    if (lineLen < (LINE_MAX - 1)) {
        lineBuf[lineLen++] = c;
    }
    // else: silently drop extra chars to remain
non-blocking
}
}
}

// Task 2: Blink LED1 according to its configured period.
void taskBlinkLED1(void) {
    unsigned long now = millis();
    unsigned long hp = halfPeriodClamp(periodMs_LED1);
    if ((now - lastToggle_LED1) >= hp) {
        state_LED1 = !state_LED1;
        digitalWrite(LED_PIN_1, state_LED1 ? HIGH : LOW);
        lastToggle_LED1 = now;
    }
}

// Task 3: Blink LED2 according to its configured period.
void taskBlinkLED2(void) {
    unsigned long now = millis();

```

```

    unsigned long hp = halfPeriodClamp(periodMs_LED2);
    if ((now - lastToggle_LED2) >= hp) {
        state_LED2 = !state_LED2;
        digitalWrite(LED_PIN_2, state_LED2 ? HIGH : LOW);
        lastToggle_LED2 = now;
    }
}

// ----- Cyclic Executive
-----
typedef void (*TaskFunc)(void);

// Round-robin task list (order can be adjusted)
TaskFunc taskList[] = {
    taskSerialInput,
    taskBlinkLED1,
    taskBlinkLED2
};

static const int NUM_TASKS = (int)(sizeof(taskList) /
sizeof(taskList[0]));

void setup() {
    pinMode(LED_PIN_1, OUTPUT);
    pinMode(LED_PIN_2, OUTPUT);

    digitalWrite(LED_PIN_1, LOW);
    digitalWrite(LED_PIN_2, LOW);

    Serial.begin(9600);
    while (!Serial) { /* wait for native USB; UNO returns
immediately */ }

```

```
    Serial.println(F("System ready (Cyclic Executive)."));
    promptAskLED();
}

void loop() {
    // Non-preemptive round-robin dispatcher
    for (int i = 0; i < NUM_TASKS; ++i) {
        taskList[i]();
    }
    // No delay(); fast polling keeps everything responsive.
}
```

Software Architecture

- Timer2 configured in CTC mode with OCR2A = 124 and prescaler = 64 produces a 2 kHz tick.
- Eight ticks form a 250 Hz PWM cycle, used to generate discrete duty levels.
- ISR increments an 8-slot counter and sets D9 HIGH when `counter < g_pwmOnTicks`.
- CE tasks include:
 - `taskSerialInput`
 - `taskBlinkLED1`
 - `taskBlinkLED2`
 - `taskButtonUpdate`

No blocking code is used, and the CE remains responsive throughout.

TESTING

Procedure

1. Verified pushbutton detection via Serial Monitor; duty steps printed correctly.
2. Attached a test LED to D9; LED brightness increased and decreased according to duty cycle.
3. Verified the motor spun correctly when powered directly from the battery pack.
4. Connected the MOSFET driver and confirmed the motor operated only when duty > 0.
5. Verified motor speed increased at 2/8, 4/8, 6/8, and reached full power at 8/8.
6. Verified CE LED blinking tasks remained responsive during motor operation.

Results

- Motor does not respond to power
- No thermal issues or instability observed.
- PWM generation is stable and consistent.

Demo Video

- <https://drive.google.com/file/d/17711eM9mdMwEbn8iZz-HK0wPUuZ5R-gE/view?usp=sharing>